Recall-Oriented Detection of Risky Clauses in Commercial Contracts by Fine-Tuning Legal-BERT on CUAD

Pranjal Jotwani
Roll No. I028
[Department name]
[Institute name, City, Country]
[email address]

Neeharika Pai
Roll No. I043
[Department name]
[Institute name, City, Country]
[email address]

***Abstract*—Commercial contracts hide clauses such as liability caps, exclusivity, non-compete and assignment restrictions that a signer must notice, and missing one costs far more than reading an extra sentence. We treat risky-clause detection as recall-oriented multi-label sentence classification and fine-tune Legal-BERT, a BERT-base encoder pre-trained on legal text, on the Contract Understanding Atticus Dataset (CUAD). Contracts are split 70/15/15 at the contract level to avoid leakage. Severe class imbalance is handled with negative down-sampling and a positive-class-weighted binary cross-entropy loss, the model is selected by validation macro F2, and a separate decision threshold is tuned for each category to maximise F2. Trained for three epochs on a single GPU over the 20 most frequent CUAD categories, the classifier reaches a validation macro F2 of 0.710 and a test macro recall of 0.78 to 0.82 depending on the label version. Combined with a lightweight regex rule layer by logical OR, the system surfaces 80.3% of the 1,524 annotated clauses in 77 unseen contracts while flagging only 21.8% of all sentences, and macro recall over 35 categories rises from 0.464 (model alone) and 0.349 (rules alone) to 0.705. On three synthetic contracts with an answer key, 45 of 48 planted clauses are caught.**

***Index Terms*—deep learning, transfer learning, Legal-BERT, multi-label classification, class imbalance, threshold calibration, contract analysis, CUAD.**

I. Introduction

Reviewing a commercial contract is slow, and the clauses that matter most are often a single sentence buried in a long document: an automatic renewal, a cap on liability, an exclusivity grant or a ban on assignment. Lawyers and small businesses therefore benefit from a tool that points at these sentences. For such a tool the two kinds of error are not equal. A false alarm costs the reader a few seconds, while a missed clause can cost money or legal exposure. We therefore design every stage of the system for high recall and judge it by how many important clauses it surfaces and how much text the reader must still check.

Pre-trained Transformer encoders such as BERT [3] have become the standard approach for sentence classification, and domain-adapted variants such as Legal-BERT [2] improve results on legal text [5]. The Contract Understanding Atticus Dataset (CUAD) [1] provides lawyer-annotated spans for 41 clause types in 510 real contracts, which makes supervised training of such a model possible. However, CUAD is extremely imbalanced at the sentence level: most sentences carry no label, and rare categories have only a few dozen training examples.

This work focuses on the deep learning component of a contract-review system. Our contributions are: (i) a conversion of CUAD into a sentence-level multi-label dataset with a contract-level train/validation/test split; (ii) end-to-end fine-tuning of Legal-BERT with imbalance-aware training (negative down-sampling and weighted binary cross-entropy), F2-based model selection and per-category threshold calibration; (iii) an ensemble with a rule layer that recovers categories the network does not cover; and (iv) an evaluation that reports clause-level precision and recall, the share of important clauses surfaced, the reading load, and an error analysis. Natural-language parts of the wider system (text extraction, sentence splitting, regex rules and an abstractive BART [7] summary) are described only briefly.

II. Related Work

BERT [3] pre-trains a bidirectional Transformer [4] encoder with masked language modelling and is fine-tuned for downstream tasks by adding a small task head. Chalkidis *et al.* [2] showed that further pre-training on legal corpora (Legal-BERT, trained on about 12 GB of English legislation, court cases and contracts) helps on legal tasks, and the LexGLUE benchmark [5] confirmed that legal-domain encoders are strong baselines for legal classification. Hendrycks *et al.* [1] released CUAD and framed clause extraction as extractive question answering; we instead frame it as sentence classification, which suits a tool that must list whole clauses verbatim. For multi-label problems with skewed labels, choosing a separate threshold per label to maximise an F-measure is a well-studied alternative to a fixed 0.5 cut-off [13]. We train with AdamW [6], mixed precision [9] and the Hugging Face Transformers library [12].

III. Dataset and Problem Formulation

A. CUAD

CUAD v1 [1] contains 510 commercial contracts (for example licence, distribution, supply and service agreements) with more than 13,000 spans annotated by legal experts in 41

categories. It is distributed in SQuAD-style JSON: each contract has a context string and, for every category, a list of answer spans with character offsets.

B. Sentence-Level Multi-Label Formulation

Each contract is split into sentences (clauses) with a legal-aware splitter that protects abbreviations and clause numbers such as "3.2.1". A clause receives category c when its character range overlaps an answer span of c, so one clause can carry several labels. Clauses shorter than four words are removed. The task is then multi-label classification: for a clause x, predict a binary vector $y \in \{0,1\}^C$, where C is the number of categories. The full table contains 114,522 clauses from the 510 contracts.

Because the network has a fixed output layer, we train on the $C = 20$ most frequent categories. The remaining categories, including four with clear risk value (Renewal Term, Uncapped Liability, Liquidated Damages, Most Favored Nation), have no output head and are handled only by the rule layer described in Section IV-F.

C. Contract-Level Split

Sentences from the same contract share parties, templates and wording, so a random sentence-level split would leak information from training to test. We therefore shuffle contract identifiers with a fixed seed (42) and split them 70/15/15 into 357 training, 76 validation and 77 test contracts. The split is saved to disk so that training, threshold tuning and evaluation always use the same contracts. The test contracts contain 15,004 clauses, of which 1,524 carry at least one CUAD label, which shows how sparse the positive class is.

IV. Methodology

A. Model Architecture

We use nlpaueb/legal-bert-base-uncased [2], which has the BERT-base architecture: 12 Transformer layers, hidden size 768, 12 attention heads and about 110 million parameters. A clause is tokenised with the model's WordPiece vocabulary and truncated to 256 tokens. The final hidden state of the [CLS] token, $h \in \mathrm{R}^{768}$, passes through dropout ($p = 0.1$) and a linear layer, and a sigmoid gives an independent probability for each category (Fig. 1):

$$p = \sigma(W h + b), \quad W \in \mathrm{R}^{20\times768} \tag{1}$$

Unlike softmax, the sigmoid lets one clause belong to several categories or to none. The whole network, not only the head, is fine-tuned.

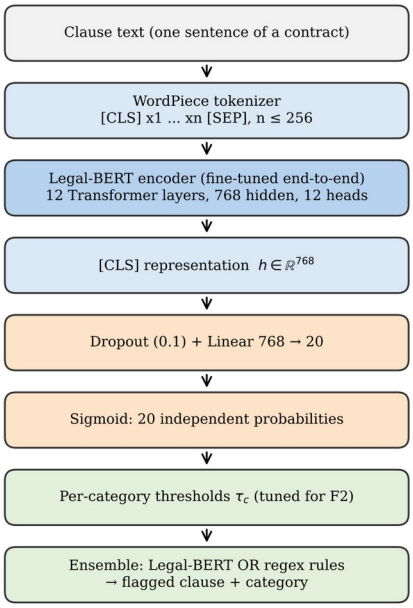


Fig. 1. Clause classifier. Every clause is encoded by Legal-BERT; the [CLS] vector feeds a linear layer with one sigmoid output per category. All encoder weights are updated during fine-tuning.

B. Handling Class Imbalance

Two complementary measures are used. First, clauses with none of the 20 labels are down-sampled in the training split only, keeping two unlabelled clauses for every labelled one. Validation and test sets are left untouched so that the reported metrics reflect the true class balance. Second, the loss is the binary cross-entropy averaged over categories, with a weight w_c on the positive term:

$$L = -(1/C)\,\Sigma_c\,[\, w_c\, y_c \log p_c + (1 - y_c) \log(1 - p_c)\,] \tag{2}$$

$$w_c = \mathrm{clip}(\,(N - n_c) / n_c,\, 1,\, 10\,) \tag{3}$$

where N is the number of training clauses and n_c the number labelled with c. Rare categories therefore receive larger weights, and the cap of 10 prevents unstable gradients for the rarest ones. In PyTorch this is BCEWithLogitsLoss with pos_weight, which also improves numerical stability by working on logits.

C. Optimisation

Training uses AdamW [6] with learning rate 2×10^{-5} and weight decay 0.01, a linear warm-up over the first 10% of steps followed by linear decay to zero, batch size 32, gradient-norm clipping at 1.0 and fp16 mixed precision [9] on the GPU. Training runs for three epochs. A checkpoint (model, optimiser, scheduler and gradient-scaler state) is written after every epoch so that a run interrupted on the shared college GPU can resume. The training loop is written in plain PyTorch rather than a high-level trainer so that every step is explicit. Table I summarises the settings.

TABLE I
TRAINING CONFIGURATION

Setting	Value
Base model	legal-bert-base-uncased (110M)
Output heads	20 sigmoid units
Max sequence length	256 tokens
Batch size / epochs	32 / 3
Optimiser	AdamW, lr 2e-5, wd 0.01
LR schedule	10% linear warm-up, linear decay
Loss	BCE with pos_weight in [1, 10]
Negative sampling	2 unlabelled : 1 labelled (train)
Precision / clipping	fp16 AMP / grad-norm 1.0
Model selection	best validation macro F2
Hardware	single CUDA GPU, ~8.7 min/epoch

D. Model Selection with F2

Since recall matters more than precision, we use the F-beta score with $\beta = 2$, which weights recall twice as heavily as precision:

$$F_2 = 5\,P\,R / (4\,P + R) \quad (4)$$

After each epoch the model is evaluated on the validation contracts at threshold 0.5, and the epoch with the highest macro F2 (averaged over the 20 categories) is kept.

E. Per-Category Threshold Calibration

A single threshold of 0.5 is a poor choice when categories differ in frequency and in how confident the model is. For each category c we search the grid $G = \{0.05, 0.10, ..., 0.50\}$ on the validation contracts:

$$\tau_c = \text{argmax}_{\tau \in G} F_{2,c}(\tau) \quad (5)$$

A clause is assigned category c when $p_c \geq \tau_c$. The tuned thresholds range from 0.05 (Parties) to 0.5, and dates and Cap On Liability receive low thresholds (0.15 to 0.2), which trades precision for recall exactly where the model is less confident.

F. Ensemble with Rules and a Positional Filter

A rule layer of 29 regular-expression categories (132 patterns), written from the training contracts, flags clauses by keyword patterns. Rule names are mapped to CUAD categories, and the final decision is the logical OR of the network and the rules. The rules mostly matter for categories the network has no head for, while the network catches paraphrases the rules miss. We also apply one positional filter: the network's Parties prediction is kept only in the first five clauses of a document, where party definitions appear (Section VII explains why this is needed).

The wider system also extracts text from PDF files, detects section headings, and writes a section-wise abstractive summary with a pre-trained BART model [7] used only for inference. Risky clauses are always shown verbatim, so the summariser can never hide one.

V. EXPERIMENTAL SETUP

Training ran once on a CUDA GPU in the college laboratory (about 26 minutes in total). Inference and evaluation ran on a laptop CPU. We report per-category precision (P), recall (R), F1 and F2, and their macro averages (the mean over categories, so rare categories count as much as common ones) and micro averages (pooled over all clause-category pairs). We also report two system-level measures: the share of the 1,524 labelled test clauses that are flagged for at least one category (*surfaced*), and the share of all test clauses that are flagged (*reading load*).

Two label versions exist. The model was trained on an early version of the clause table in which labels were assigned by substring matching. We later rebuilt the labels from the answers' character offsets, which are more accurate, and all results in Tables III to VI use these corrected labels. Results on the original labels, computed at the end of the training run, are given for completeness.

VI. RESULTS AND DISCUSSION

A. Training Dynamics

Table II and Fig. 2 show the training loss falling from 0.175 to 0.036 over three epochs while validation macro F2 rises from 0.663 to 0.710. Validation recall peaks at epoch 2 (0.808) and precision improves in epoch 3 (0.556), so epoch 3 has the best F2 and was kept. The gap between the falling training loss and the slowly rising validation scores suggests that more epochs would mainly increase over-fitting, so three epochs is a reasonable budget for this data size.

TABLE II
TRAINING HISTORY (VALIDATION MACRO SCORES AT 0.5)

Epoch	Train loss	P	R	F1	F2
1	0.175	0.502	0.742	0.585	0.663
2	0.054	0.496	0.808	0.602	0.704
3	0.036	0.556	0.778	0.637	**0.710**

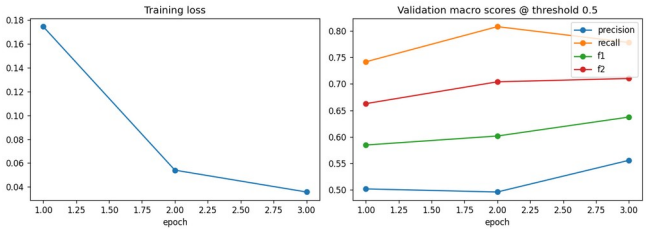


Fig. 2. Training loss (left) and validation macro precision, recall, F1 and F2 at threshold 0.5 (right) for each epoch.

B. Classifier Performance on Test Contracts

On the 77 held-out contracts with the original labels, the classifier with tuned thresholds reaches macro precision 0.530, recall 0.779 and F1 0.621. The best categories are Governing Law (R 1.00, F1 0.95), Expiration Date (F1 0.81), Insurance (F1 0.77) and Anti-Assignment (F1 0.74). Table III gives the results against the corrected labels. Macro recall is 0.800 at the default threshold and 0.822 with tuned thresholds, confirming that calibration buys recall at a small cost in precision. Micro precision is low (0.12 to 0.13) for the raw model because one category, Parties, fires on thousands of clauses (Section VII); the deployed model with the positional filter raises micro

precision to 0.448 and micro F1 from 0.203 to 0.579 while keeping micro recall above 0.81.

TABLE III
TEST RESULTS ON THE 20 LEARNED CATEGORIES (MA = MACRO, MI = MICRO)

System	MaP	MaR	MaF1	MiP	MiR	MiF1
BERT @0.5	0.510	0.800	0.596	0.133	0.812	0.228
BERT tuned τ	0.471	**0.822**	0.568	0.115	0.836	0.203
+ Parties filter	0.492	0.812	0.594	0.448	0.816	0.579
Ensemble	0.439	0.818	0.552	0.399	0.823	0.537

Fig. 3 shows micro precision against recall for every threshold of the raw model, together with the operating points. The curve is flat at a low precision for most of its range, again because of the noisy Parties outputs, and the deployed points sit well above it. This shows that a cheap, domain-motivated post-processing step can matter as much as the threshold itself.

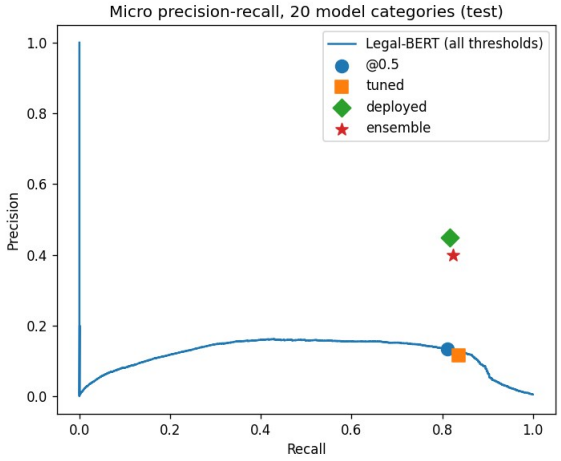


Fig. 3. Micro precision-recall curve of the raw Legal-BERT outputs on the 20 learned categories (test), with the operating points at 0.5, with tuned thresholds, as deployed, and for the ensemble.

C. Per-Category Analysis

Fig. 4 gives the test recall of every evaluated category. Among the 20 learned categories, recall ranges from 0.60 (Revenue/Profit Sharing) to 0.985 (Governing Law). Categories with formulaic wording (governing law, insurance, dates, anti-assignment, termination for convenience) are above 0.9. Categories whose meaning depends on a commercial arrangement rather than fixed phrases, such as Revenue/Profit Sharing, Rofr/Rofo/Rofn and Post-Termination Services, are hardest at 0.60 to 0.63.

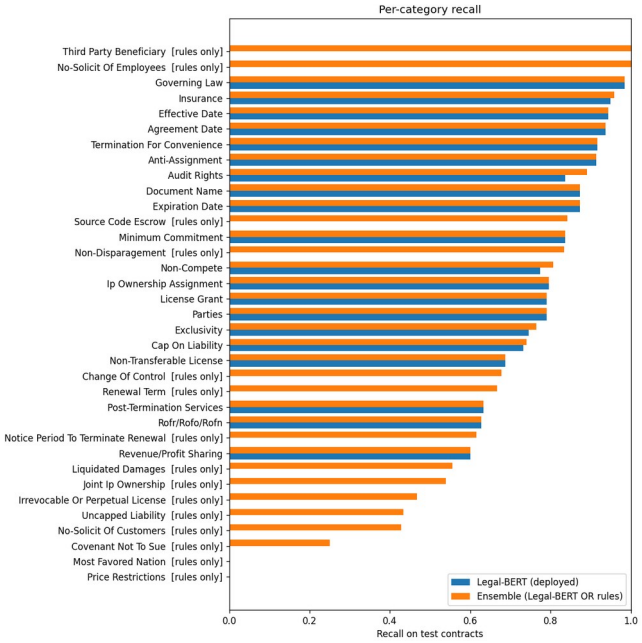


Fig. 4. Test recall per category for the deployed Legal-BERT (blue) and the ensemble (orange). Categories marked "rules only" have no network output head.

Fig. 5 plots recall against the number of training clauses. There is no clear trend: Termination For Convenience reaches 0.92 recall with the fewest examples, while Revenue/Profit Sharing has about 330 and reaches only 0.60. Within the data range of CUAD, how consistently a category is worded matters more than how many examples it has, which supports the use of a pre-trained legal encoder over training from scratch.

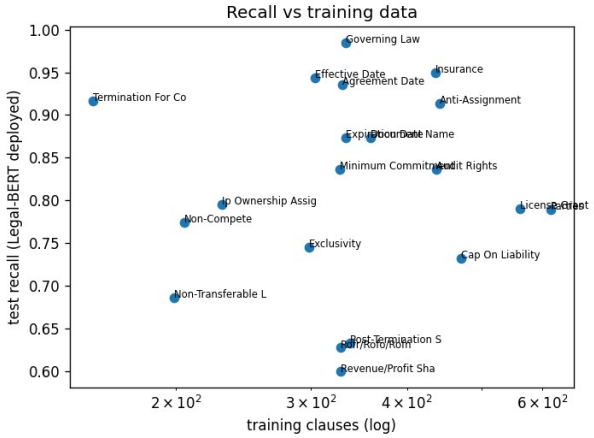


Fig. 5. Test recall of the deployed Legal-BERT against the number of training clauses per category (log scale).

D. Network versus Rules and the Ensemble

Table IV compares the detectors. On the eight categories that both the network and the rules cover, Legal-BERT has much higher macro recall than the rules (0.833 against 0.487) and higher precision, so the learned model dominates hand-written patterns where training data exist. The OR ensemble adds a little recall (0.848) at a cost in precision. Over all 35 evaluated

categories, the network alone reaches only 0.464 macro recall because it has no head for 15 of them; the ensemble raises macro recall to 0.705. Rules-only categories average 0.55 recall against 0.82 for categories with a network head, which shows that the remaining gap is mainly a coverage problem of the network rather than of the rules.

TABLE IV
RULES, LEGAL-BERT AND ENSEMBLE (TEST, MACRO AVERAGES)

Categories	System	P	R	F1
8 shared	Rules	0.435	0.487	0.442
8 shared	Legal-BERT	0.540	0.833	0.645
8 shared	Ensemble	0.409	**0.848**	0.540
All 35	Rules	0.232	0.349	0.255
All 35	Legal-BERT	0.281	0.464	0.339
All 35	Ensemble	0.384	**0.705**	0.470

E. Clause Surfacing and Reading Load

Table V gives the system-level view. Of the 1,524 labelled clauses in the test contracts, the ensemble flags 80.3% while flagging only 21.8% of all sentences, so a reader who checks the flagged fifth of a contract sees four out of five important clauses. Legal-BERT alone surfaces 73.7% and the rules alone 35.5%. Without the Parties filter the reading load would be 63.8%, which would defeat the purpose of the tool.

TABLE V
IMPORTANT CLAUSES SURFACED ON TEST CONTRACTS

Measure	Value
Labelled clauses (any CUAD label)	1,524
Surfaced by rules alone	35.5%
Surfaced by Legal-BERT alone (deployed)	73.7%
Surfaced by ensemble (deployed)	**80.3%**
Reading load: share of all clauses flagged	**21.8%**
Reading load without Parties filter	63.8%

F. End-to-End Test on Synthetic Contracts

As a sanity check of the complete pipeline from PDF to flagged clauses, we wrote three synthetic contracts (a mutual NDA, a residential lease and SaaS terms) containing 48 planted risky clauses and 8 benign ones, with an answer key. The system caught 45 of the 48 planted clauses (93.8%) and assigned the correct category to 43 (89.6%), with one false alarm among the 8 benign clauses. Across the three documents, 13 flagged clauses came from the network alone, 17 from the rules alone and 17 from both, so both detectors contribute. This set is small and was written by us, so it shows that the pipeline works end to end rather than measuring accuracy.

For completeness, the abstractive summariser (pre-trained BART, inference only) scores ROUGE-1 0.403 against 0.213 for a Lead-3 baseline on 50 BillSum [10] documents, using ROUGE [11].

G. Error Analysis

Table VI groups the missed labels by clause length. Very short clauses of up to 15 tokens are missed almost half the time (46.7%), since a fragment such as a date or a heading carries little context for the encoder. The miss rate is lowest for 31 to 120 tokens (about 17%) and rises again above 120 tokens (21.9%), partly because clauses longer than 256 WordPiece tokens are truncated. Manual inspection of high-confidence false positives showed many clauses that are relevant but were not highlighted by the CUAD annotators, so part of the measured precision loss is label noise rather than model error.

TABLE VI
MISSED LABELS BY CLAUSE LENGTH (ENSEMBLE, TEST)

Length (tokens)	Labels	Missed	Miss rate
≤ 15	107	50	0.467
16–30	385	81	0.210
31–60	670	116	0.173
61–120	453	77	0.170
> 120	155	34	0.219

VII. LIMITATIONS

Twenty output heads. The network was trained on the 20 most frequent categories only. Renewal Term, Uncapped Liability, Liquidated Damages and Most Favored Nation have no head and depend on rules with recall between 0.00 and 0.67. The training code now supports all 41 categories, but there was no further GPU access to retrain.

Noisy Parties labels. The early label version marked a clause with a category if a short answer string such as a party name appeared anywhere in it, which produced far too many Parties labels. The model learned this, so at its tuned threshold of 0.05 the raw Parties output flags 8,787 test clauses with precision 0.018. The positional filter reduces this to 288 flags with precision 0.431 and recall 0.790. Retraining on the corrected labels is the proper fix.

Other limits. Results come from a single training run and seed, so we cannot report variance. Clauses are truncated to 256 tokens and classified without document context. The synthetic test set is small, and no real-world contracts were labelled for testing. The tool is not legal advice.

VIII. CONCLUSION AND FUTURE WORK

We fine-tuned Legal-BERT as a multi-label sentence classifier for risky contract clauses on CUAD, using a contract-level split, imbalance-aware training, F2-based model selection and per-category threshold calibration. Trained for three epochs on one GPU, the model reaches validation macro F2 0.710 and test macro recall of about 0.8 on its 20 categories, and it clearly outperforms hand-written rules where both apply (0.833 against 0.487 macro recall). The OR ensemble with rules extends coverage to 35 categories and surfaces 80.3% of important test clauses while asking the reader to check only 21.8% of the text.

The main next step is to retrain on the corrected labels with all 41 output heads, which should remove the Parties filter and the rules-only gap. Further directions are a long-context encoder such as Longformer [8] or a sliding window to avoid truncation and add document context, focal or asymmetric losses for the

rarest categories, several seeds to measure variance, and evaluation on a labelled set of real contracts.

References

[1] D. Hendrycks, C. Burns, A. Chen, and S. Ball, "CUAD: An expert-annotated NLP dataset for legal contract review," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.

[2] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, "LEGAL-BERT: The muppets straight out of law school," in *Findings of EMNLP*, 2020, pp. 2898–2904.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[4] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017, pp. 5998–6008.

[5] I. Chalkidis *et al.*, "LexGLUE: A benchmark dataset for legal language understanding in English," in *Proc. ACL*, 2022, pp. 4310–4330.

[6] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[7] M. Lewis *et al.*, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proc. ACL*, 2020, pp. 7871–7880.

[8] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," arXiv:2004.05150, 2020.

[9] P. Micikevicius *et al.*, "Mixed precision training," in *Proc. ICLR*, 2018.

[10] A. Kornilova and V. Eidelman, "BillSum: A corpus for automatic summarization of US legislation," in *Proc. Workshop on New Frontiers in Summarization*, 2019, pp. 48–56.

[11] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, 2004, pp. 74–81.

[12] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45.

[13] Z. C. Lipton, C. Elkan, and B. Naryanaswamy, "Optimal thresholding of classifiers to maximize F1 measure," in *Proc. ECML PKDD*, 2014, pp. 225–239.